CAS (Compare-And-Swap) is a **hardware-level atomic operation** that lets multiple threads update shared memory **without locks**. It's the core trick behind most **lock-free** data structures in Java.

The intuition (human version)

> "Update this value **only if** nobody else changed it since I last looked."

If someone *did* change it → you **retry**, not block.

The formal definition

CAS works on **one memory location** and takes **three values**:

```
CAS(memoryLocation, expectedValue, newValue)
```

- If `memoryLocation == expectedValue`
  - write `newValue`
  - return `true`
- Else
  - do nothing
  - return `false`

💥 **The entire check + update happens atomically in hardware**

Why CAS is special

- ✅ No locks
- ✅ No blocking
- ✅ No context switches
- ✅ Extremely fast under low-moderate contention
- ❌ May retry under high contention

CAS in Java (what you actually use)

You don't call CAS directly. Java exposes it via:

- `AtomicInteger`, `AtomicLong`, `AtomicReference`
- `VarHandle`
- Internally: `Unsafe.compareAndSwapXxx`

Example:

```
AtomicInteger count = new AtomicInteger(0);

// CAS loop
while (true) {
    int current = count.get();
    int next = current + 1;
    if (count.compareAndSet(current, next)) {
        break; // success
    }
}
```

That `compareAndSet` **is CAS**.

What happens at CPU level

1. Thread reads value → `V`
2. CPU executes CAS instruction:
    - compares memory with `V`
    - swaps if equal
3. CPU ensures:
    - atomicity
    - memory visibility
    - ordering guarantees

On x86 this is typically:

- `CMPXCHG` instruction

  On ARM:
- `LDXR / STXR`

CAS retry loop (the lock-free pattern)

```
read → compute → CAS
          ↑
          └── retry if failed
```

This is called **optimistic concurrency**:

- Assume success
- Retry only on conflict

CAS vs locks (why lock-free scales better)

Aspect	Lock	CAS
Blocking	Yes	No
Context switches	Possible	Never
Deadlock risk	Yes	No
Starvation	Possible	Possible (rare)
Performance	Degrades under contention	Degrades more gracefully

---

The ABA problem (important ⚠️)

CAS checks **value equality**, not **history**.

Problem scenario

1. Thread T1 reads `A`
2. Thread T2 changes `A → B → A`
3. T1 CAS succeeds (but state changed!)

Why this is dangerous

CAS thinks *nothing changed*, but it did.

How Java solves ABA

- **Stamped references**

`AtomicStampedReference<V>`

- **Versioning**

`(value, version)`

If version changes → CAS fails.

Why CAS is still safe in Java collections

Because they:

- Use **indirection**
- Add **markers**
- Use **lazy deletion**
- Combine CAS with **retry + cleanup**

Example:

- `ConcurrentLinkedDeque`
- `ConcurrentHashMap`
- `Atomic*` classes

CAS and the Java Memory Model

A successful CAS:

- Has **volatile read + write semantics**
- Establishes **happens-before**
- Guarantees visibility across threads

So this is safe:

```
atomicRef.compareAndSet(old, new);
// other threads will see `new`
```

When CAS shines ✨

✅ High concurrency
✅ Short critical sections
✅ Read-mostly workloads
✅ Lock-free algorithms

When CAS struggles

❌ Heavy contention → spin retries
❌ Complex multi-variable invariants
❌ When fairness is required

(That's why some structures still use locks.)

One-line interview answer

> **CAS is a hardware-atomic instruction that updates a value only if it hasn't changed, enabling lock-free concurrency via retry loops instead of blocking.**

If you want next:

- CAS vs `synchronized`
- How CAS enables **lock-free stacks & queues**
- Why **spinning is still better than blocking**
- Deep dive into **VarHandle vs Atomic classes**

Just say where you want to go 🚀